Table of Contents

Introduction **2**

Http Get method **3**

Run the backend and frontend simultaneously **7**

Run with two terminals: *7*

Run with one terminal *7*

Optional: Delete the Vite Boilerplate CSS (Standard Practice) *8*

Optional: React Query Devtools **9**

Demo: Http Get

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini.

Introduction

In this demo, we shift from using mock data to fetching real data from a backend API for the ItemListPage component using **TanStack Query** (useQuery).

Key Concepts:

1. **AJAX (Asynchronous JavaScript and XML):** A technique that allows for asynchronous data retrieval from a server without reloading the entire page, providing a dynamic user experience.
2. **TanStack Query Setup:**
    - **QueryClient:** Manages and holds the query cache for the entire application.
    - **QueryClientProvider:** Wraps the application structure to make the QueryClient instance accessible via React Context.
3. **useQuery Hook:** A specialised hook that replaces manual state and effect management by handling fetching, caching, loading states, error handling, and manual refetching. Key features include:
    - **queryKey:** A unique array identifier used internally for caching and invalidating data.
    - **queryFn:** The asynchronous function (e.g., fetchItems) responsible for making the network request.
    - **State Management:** Exposes reactive values like data, isLoading, error, and utilities like refetch.
4. **Data Fetching & Verification:**
    - Sending HTTP GET requests via the Fetch API.
    - Explicitly checking response.ok to handle HTTP error status codes outside the 200–299 range.
5. **UI & Data Presentation:**
    - Conditional UI updates based on loading states, error handling, and dynamic table rendering.
    - Prepending backend server base URLs (API_URL) to access static media assets (such as images stored in wwwroot).
6. **Concurrent Development Setup:**
    - Running both frontend and backend projects simultaneously in a single terminal using the concurrently package with automated process termination (-k) and clean exit handling (-s first).
7. **React Query Devtools:** Install `@tanstack/react-query-devtools` to easily inspect, debug, and monitor query statuses, cache states, and fetched data in development without using console logs.

Http Get method

AJAX stands for **Asynchronous JavaScript and XML**. It is a technique used to send and receive data from a server asynchronously without needing to reload the entire page. This allows web applications to update parts of the web page dynamically, providing a smoother and more interactive user experience. AJAX is the historical term for fetching server data without reloading the browser, but today we simply refer to this as making **asynchronous HTTP requests**.

Previously, we used mock data for the front-end ItemListPage. This time, we are going to fetch data from the backend API. We adopt the modern practice of **useQuery** from the **TanStack Query** library, which is a dedicated state-management library that handles fetching, caching, background re-fetching, loading states, and error handling automatically, eliminating the need for manual useEffect hooks and local state management.

First install the tanstack library under your frontend project

```
npm install @tanstack/react-query
```

Then change App.jsx as below

App.jsx U

6-React-Intro > Demo-react-3-httpget > myshop > src > App.jsx > ...

```jsx
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import Container from 'react-bootstrap/Container'
import HomePage from './home/HomePage'
import ItemListPage from './items/ItemListPage'
import NavMenu from './shared/NavMenu'
import './App.css'

const queryClient = new QueryClient()

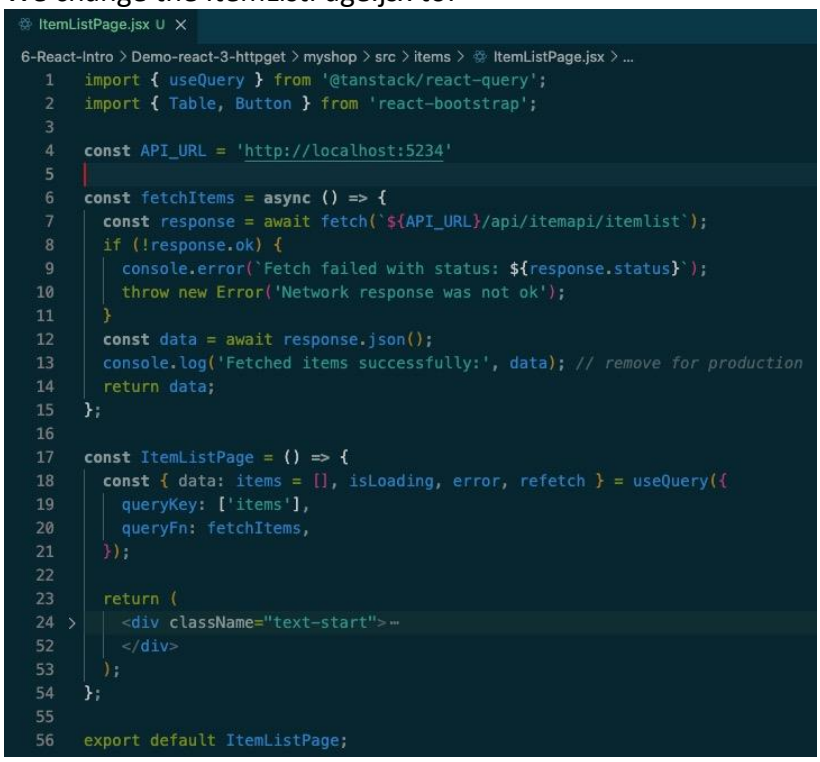
function App() {

  return (
    <QueryClientProvider client={queryClient}>
    <Container>
    <NavMenu />
        <Router>
            <Routes>
              <Route path="/" element={<HomePage />} />
              <Route path="/items" element={<ItemListPage />} />
              <Route path="*" element={<Navigate to="/" replace />} />
            </Routes>
        </Router>
    </Container>
    </QueryClientProvider>
  )
}

export default App
```

QueryClient: An object that manages and holds the query cache for your entire application.
QueryClientProvider: A component used to wrap your application structure, making the QueryClient instance accessible to all child components via React Context.

We change the ItemListPage.jsx to:



useQuery Hook: A specialised hook used to handle asynchronous data fetching. Key configuration properties and returned values include:

- **queryKey**: A unique key (passed as an array) that **TanStack** Query uses internally for caching, invalidating, and refetching this specific dataset.
- **queryFn**: The asynchronous function responsible for executing the network request and returning the data.
- **State Management**:
- **data**: The dataset returned by the query function (aliased as items or given default values [] in the component). It stores the list of items fetched from the API.
- **isLoading**: A boolean flag provided directly by TanStack Query indicating whether the data loading request is in progress.
- **error**: Captures and holds any exception thrown inside the fetch function during the request.
- **refetch**: A utility function provided by TanStack Query to manually trigger a fresh network request on demand.

fetchItems Function:

- Uses the JavaScript Fetch API to send an HTTP GET request to the backend API endpoint.
- Checks response.ok to verify the HTTP response status. If the status falls outside the success range (200-299), it explicitly throws an error so TanStack Query can register the request failure.
- Parses and returns the JSON payload, which TanStack Query automatically catches and assigns to the data return value.

```
<div className="text-start">
  <h1>Items</h1>
  <Button onClick={() => refetch()} className="btn btn-primary mb-3" disabled={isLoading}>
    {isLoading ? 'Loading...' : 'Refresh Items'}
  </Button>
  {error && <p style={{ color: 'red' }}>Failed to fetch items.</p>}
  <Table striped bordered hover>
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Price</th>
      <th>Descriptions</th>
      <th>Images</th>
    </tr>
  </thead>
  <tbody>
    {items.map(item => (
      <tr key={item.itemId}>
        <td>{item.itemId}</td>
        <td>{item.name}</td>
        <td>{item.price} NOK</td>
        <td>{item.description}</td>
        <td><img src={`${API_URL}${item.imageUrl}`} alt={item.name} width="120" /></td>
      </tr>
    ))}
  </tbody>
  </Table>
</div>
```

ItemListPage UI Logic:

- **Button Component:** Sets up an event handler that triggers the **refetch** function when the button is clicked. The button dynamically disables itself whenever loading is true.
- **className="btn btn-primary mb-3"**: Applies Bootstrap classes to style the button. btn and btn-primary apply Bootstrap button styles, while mb-3 adds a margin at the bottom.
- **disabled={loading}**: Disables the button while data is being fetched (loading is true).
- **Button Label**: **{loading ? 'Loading...' : 'Refresh Items'}**: Displays "Loading..." when the loading state is true. Otherwise, it shows "Refresh Items".
- **Error Message: {error && <p style={{ color: 'red' }}>{error}</p>}**: Conditionally renders a paragraph with the error message in red if error is not null or undefined. The && operator is used here for conditional rendering.
- **Table Rendering:** Maps through the items array to dynamically generate table rows for each item ID, name, price, description, and image.
- **:** Displays an image with a source URL constructed from API_URL and item.ImageUrl. Note that the images come from the server-side, so the **API_URL** is needed before the **item.ImageUrl** to “get” the images from the **wwwroot** folder from the server-side.
- The alt attribute provides alternative text for accessibility, and the width attribute sets the image width to 120 pixels.

In React, hooks and states are fundamental concepts for managing and handling component logic and lifecycle.

Hooks are special **functions** that let you "hook into" React features such as state and lifecycle methods from functional components. Hooks allow functional components to manage state and side effects, making them more versatile and powerful.

State in React is a way to store and manage **dynamic data** in a component. It allows a component to keep track of changes and re-render itself when state updates. This will be explained further later.

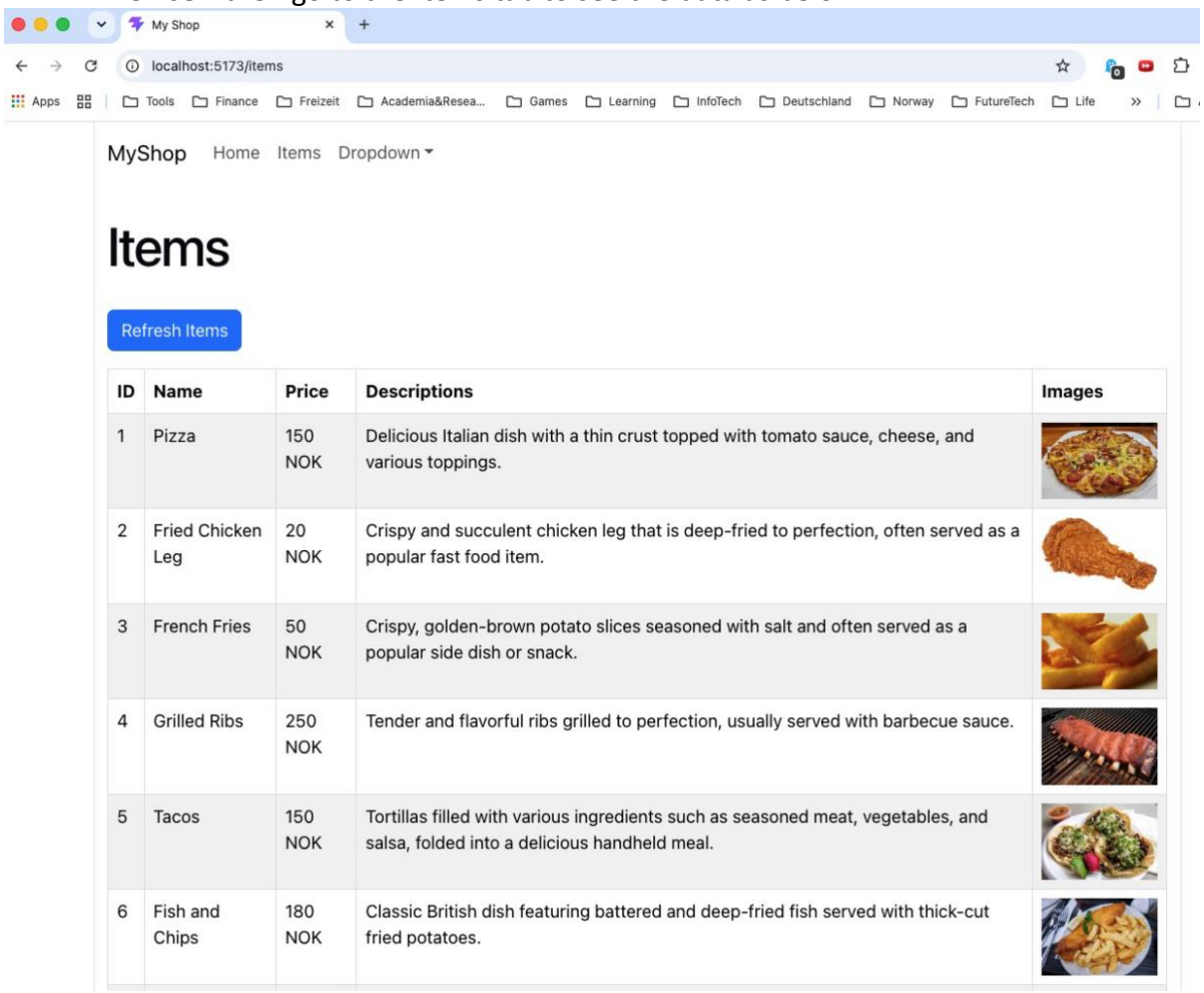
You can use inspect the console log to see the details.

Run the backend and frontend simultaneously

Run with two terminals:

Open two terminal

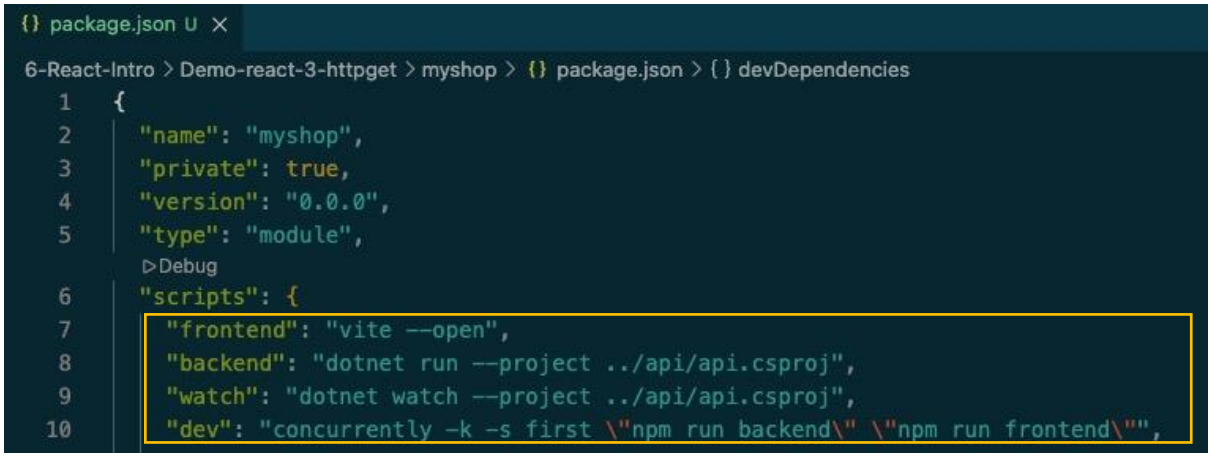
- **Terminal 1:** run the backend by cd api, then dotnet watch or dotnet run
- **Terminal 2:** run the frontend by cd myshop, then npm run dev
- **Browser:** then go to the Items tab to see the data as below



Run with one terminal

We can configure the **package.json** to run both at the same time.
First go to the front-end folder `cd myshop`, then install a tool
`npm install concurrently --save-dev`

Modify the **package.json** as below.



Individual Scripts ("frontend", "backend", "watch"): Defines clear, standalone commands. We can run either application on its own for targeted debugging (e.g., npm run backend).
concurrently: Executes both the backend and frontend servers simultaneously inside a single terminal window.
"npm:backend" / "npm:frontend": Uses concurrently's clean shorthand to reference the scripts above, avoiding messy nested strings.
Note: we do not run dotnet watch concurrently because it has an issue of using Ctrl C to terminate it together with the frontend. You can experiment replacing dotnet run with dotnet watch in backend to see the effect.
-k (Kill Others): If either application crashes or shuts down, this automatically terminates the other. This prevents orphaned background processes from holding other ports.
-s first (Success First): Ensures that pressing Ctrl + C stops both tasks cleanly and exits without throwing unnecessary error codes in the terminal.

Now run the front-end and back-end by running

```
npm run dev
```

You will see both are running.
The backend normally takes more time to start, so the frontend may start with a data fetching error. After the backend starts, click refresh to load the data again.

Optional: Delete the Vite Boilerplate CSS (Standard Practice)

When setting up Bootstrap in a new project, delete the default starter code inside index.css and App.css, or leave them empty. This prevents Vite's template styles from conflicting with Bootstrap.

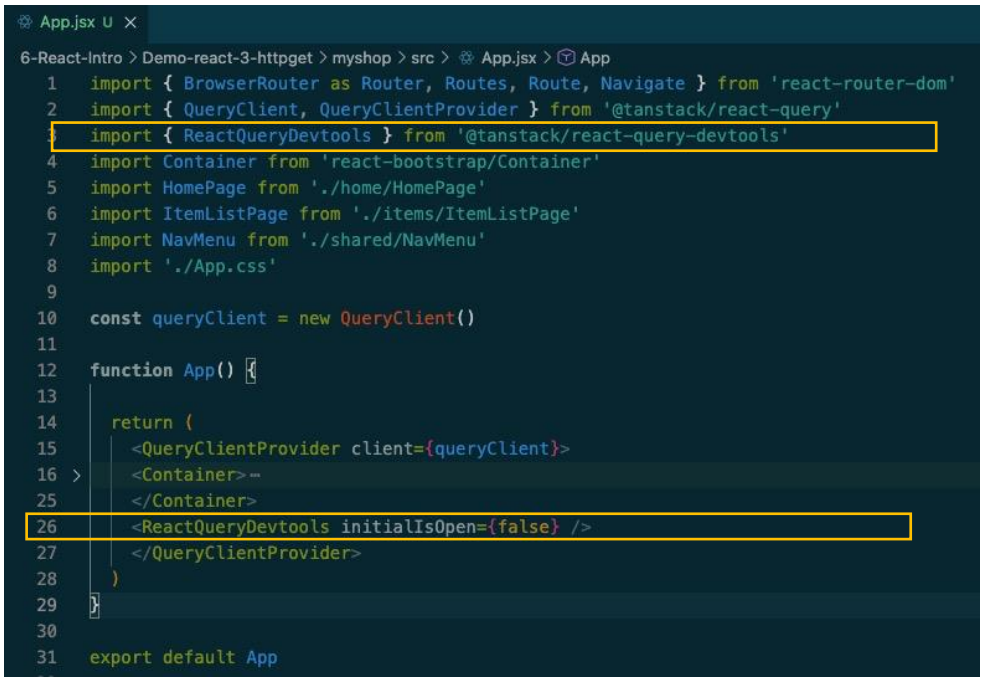
- Open src/index.css and **delete everything** inside it.
- Open src/App.css and **delete everything** inside it.

Optional: React Query Devtools

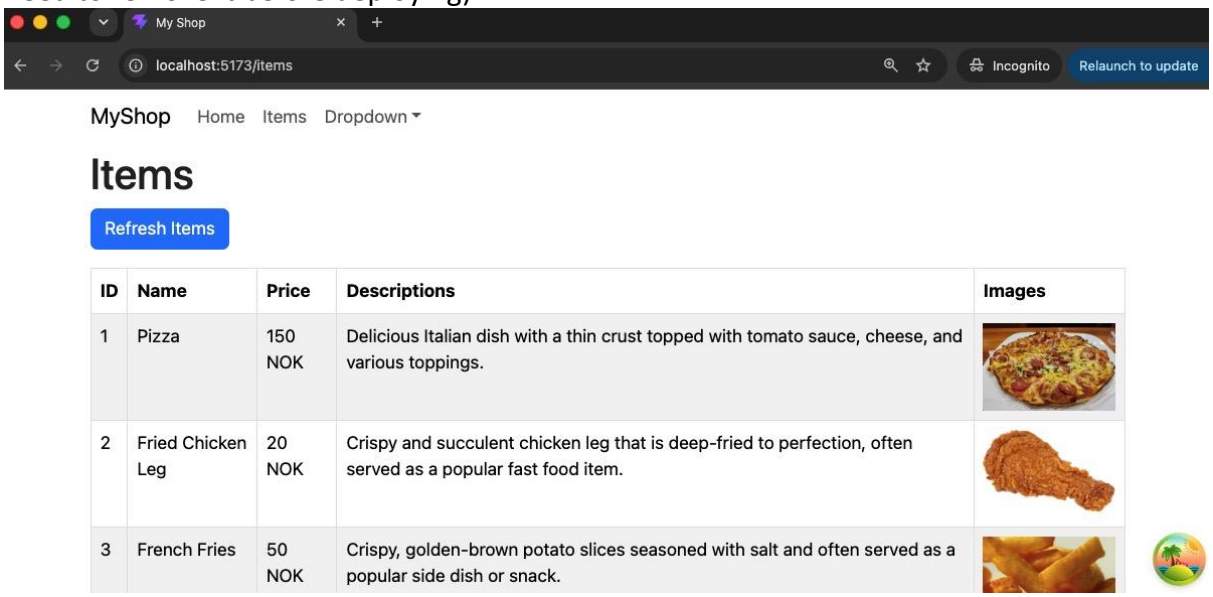
When working with `useQuery`, it's often useful to see exactly what's happening behind the scenes: is the query loading, has it succeeded, is the data stale, when will it refetch. Rather than sprinkling `console.log` calls throughout your components to answer these questions, TanStack Query provides a dedicated visual tool for this: the React Query Devtools.

Install it as a separate package:
`npm install @tanstack/react-query-devtools`

Then add it to `App.jsx`, alongside your existing `QueryClientProvider`:
`initialIsOpen={false}` means the panel starts closed, so it doesn't get in the way. You open it only when you want to inspect something. This replaces most of what a `console.log` inside `fetchItems` was previously used for. For instance, the line in ItemListPage.tsx:
`console.log('Fetched items successfully:', data); // remove for production`
is no longer necessary. You can see the exact same `data` in the Devtools panel any time, without needing to remember to remove the log later. As a general habit going forward: reach for the Devtools first when the question is *"what state is this query/mutation in?"*, and reserve `console.log`/`console.error` for logging values inside your own custom logic, where Devtools has no visibility.

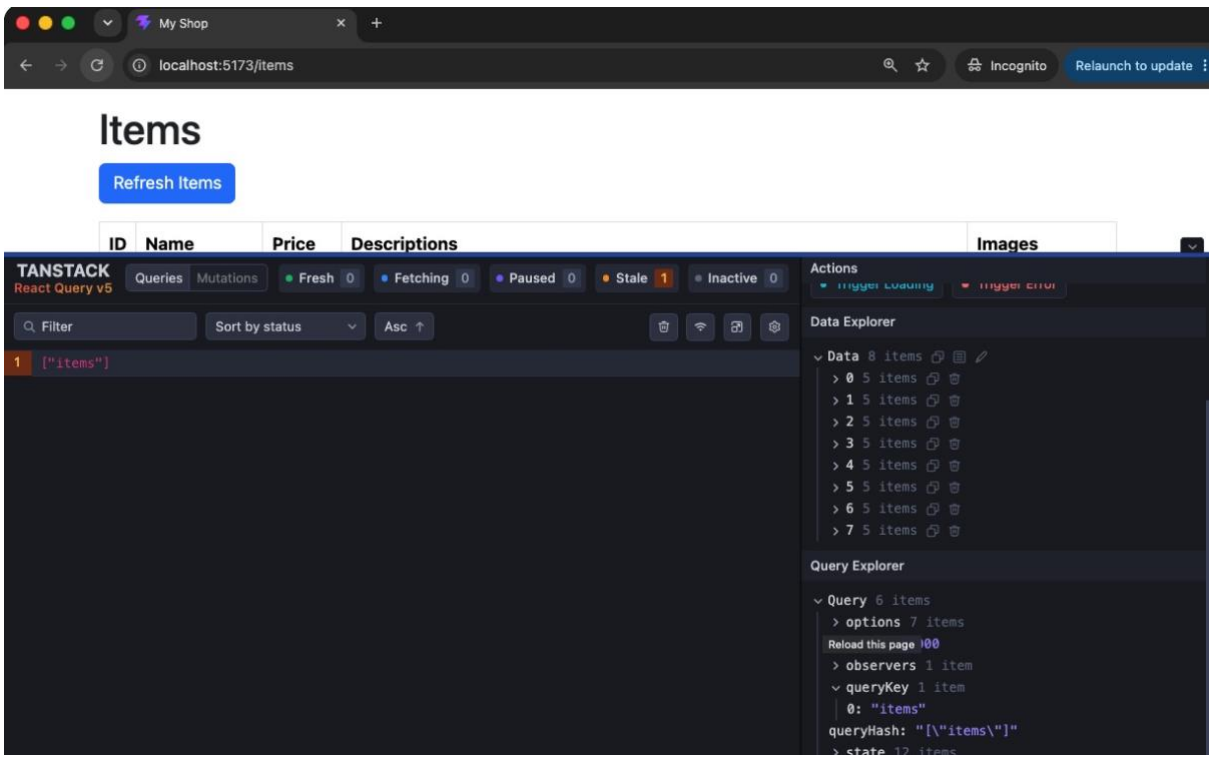


`ReactQueryDevtools` renders a small floating icon in the corner of your browser window (only in development mode. It's automatically excluded from production builds, so you don't need to remove it before deploying).



Clicking it opens a panel showing every query currently registered in your app, including:

- The query's key (e.g. ['items']), so you can immediately tell which query you're looking at.
- Its current status: fresh, fetching, stale, or inactive, shown with a colour-coded indicator.
- The actual data it holds, which you can expand and inspect, just like you would with a console.log, but without needing to add or remove any logging code.
- How long ago it last fetched, and a button to manually trigger a refetch.



Congratulations! You now know how to use Http Get from the frontend to retrieve data from the backend API. ☺